

Chapter 1: Software Project Planning

1.1 Description of the Software System under Study

NexusFlow is a web-based, role-driven Enterprise Project and Team Collaboration Management System developed as a solo BCA summer training project. Where a plain CRUD system would stop at storing projects and tasks, NexusFlow's own name states its purpose: “Nexus” — connecting people through project membership, task assignment, and threaded discussion — and “Flow” — moving work through a defined lifecycle rather than a single status field. Every task in NexusFlow now moves through an explicit workflow — created, assigned, accepted by its assignee, worked on with logged progress notes, optionally clarified through a request/response exchange with the task's creator or a manager, and finally marked complete — with every step of that journey recorded on an Activity Timeline rather than silently overwritten.

The system supports four distinct user roles — Admin, Project Manager, Team Member, and Client — each with role-specific permissions enforced through JWT-based authentication and authorization, refined further at the level of individual actions: for example, only a task's assignee may Accept it or Request Clarification on it, while only its creator or a Project Manager/Admin may respond to that clarification or move its status forward. Admins manage users and organizations; Project Managers create and oversee projects and assign tasks; Team Members work through the task lifecycle described above; Clients view project progress and reports. For the purposes of this training environment, the Project Guide, Rohit Kumar (Assistant Professor), acted as the primary user in defining and reviewing functional requirements, in line with standard summer training practice.

NexusFlow follows a three-tier client-server architecture: an Angular single-page front end communicates with an ASP.NET Core (.NET 10) REST API, which in turn persists data through Entity Framework Core to a Microsoft SQL Server database. The backend follows Clean Architecture principles, separating the system into four distinct layers — Domain, Application, Infrastructure, and API — to keep business logic independent of frameworks and easily testable. As of this submission, the system has reached Release Candidate v1.0 — a Review Build demonstrated to and evaluated by the Project Guide — confirming that the workflow described above runs end-to-end against a live database rather than existing only as a design.

1.2 Data Collection

Requirements and design references for NexusFlow were drawn from a combination of industry documentation and comparative study of existing project-management products:

- Official technical documentation: Microsoft Learn (ASP.NET Core, Entity Framework Core, SQL Server), Angular.dev (Angular 20 framework documentation), and the AutoMapper/FluentValidation project documentation.
- Comparative feature study of existing commercial project-management platforms — Jira, Trello, Asana, and Microsoft Planner — to identify the core module set (project, task, comment, notification, dashboard) common to enterprise-grade collaboration tools.
- Workflow-pattern research: task-lifecycle and audit-trail conventions observed in Jira, Linear, and GitHub Issues (Open → Accepted → In Progress → Ready for Review → Completed, with a chronological activity feed) informed the design of NexusFlow's own task workflow and Activity Timeline.
- Institutional guidance: IITM's Summer Training Project Guidelines (Appendix A/B) and the TuxAcademy/Linux Mantra Full Stack Development training curriculum, which structured the five-week technology ramp-up (Web Fundamentals → Angular → ASP.NET Core APIs → SQL Server → Integration & Deployment) preceding project development.

The functional modules selected for study and implementation — Authentication, User Management, Organization Management, Project Management, Task Management & Workflow, Comment & Collaboration, Notification, and Dashboard & Reporting — were finalized after this comparative review and confirmed with the Project Guide before development began.

1.3 Tools and Platform

1.3.1 Hardware Specification

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher (64-bit)
RAM	8 GB minimum (16 GB recommended for running SQL Server and Visual Studio together)
Storage	100 GB free disk space (SSD preferred)
Display	1366 x 768 resolution or higher
Operating System	Windows 10/11 (64-bit)

Table 1.1: Hardware Specification

1.3.2 Software Specification

Layer	Technology
Frontend	Angular 20, TypeScript, HTML5, CSS3, Angular Material
Backend	ASP.NET Core (.NET 10), C#, Entity Framework Core, AutoMapper, FluentValidation
Database	Microsoft SQL Server 2022 (Developer Edition)
Authentication	JSON Web Tokens (JWT)
IDE / Tools	Visual Studio 2022, Visual Studio Code, SQL Server Management Studio (SSMS)
Version Control	Git, GitHub
API Testing	Postman
Architecture	Clean Architecture (Domain, Application, Infrastructure, API layers)
Design Patterns	Repository Pattern, Unit of Work Pattern, Dependency Injection

Table 1.2: Software Specification

1.4 Project Planning

1.4.1 Work Distribution Table

NexusFlow is developed individually. The table below shows the distribution of work across the training period by phase, rather than by team member, since the project has a single developer.

Phase	Task / Activity	Duration	Owner
1	Requirement analysis, topic selection, project planning	Jun 28 – Jul 1	Harsh
2	Synopsis preparation and submission	Jul 1 – Jul 6	Harsh
3	Database design and backend core modules (Auth, Project, Task)	Jul 6 – Jul 15	Harsh
4	Remaining backend modules + AutoMapper/FluentValidation wiring	Jul 15 – Jul 20	Harsh
5	Angular frontend: Auth, Dashboard, Projects, Tasks (CRUD)	Jul 20 – Jul 22	Harsh
6	Task Workflow Engine — Accept/Clarify/Respond/Status, Activity Timeline, My Tasks & Task Details pages (Review Build v1.0)	Jul 22 – Jul 24	Harsh
7	Mentor review and feedback incorporation	Jul 24	Harsh
8	Documentation (Chapters 1–4) and final report assembly	Jul 24 – Aug 7	Harsh

Table 1.3: Team-Member Wise Work Distribution

1.4.2 PERT Chart

The PERT chart below represents the sequence of major activities undertaken during the training period, with each numbered event marking the completion of a phase and the estimated duration of the activity leading into it. Phase 6 — the Task Workflow Engine — was added as a distinct increment ahead of the Mentor Review, reflecting the actual project timeline rather than the originally-planned seven phases.

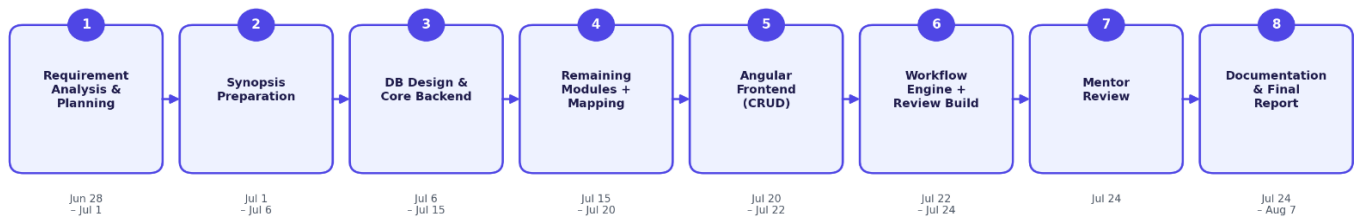


Figure 1.1: PERT Chart

1.5 Methodology

1.5.1 SDLC Model

The Incremental Model has been selected for the development of NexusFlow.

1.5.2 Justification for the Selection of Model

NexusFlow consists of multiple independent but interconnected modules — Authentication, Project Management, Task Management, Comments, Notifications, and Dashboard reporting. The Incremental Model allows each module to be designed, developed, and verified as a self-contained increment before the next begins, which proved directly applicable during actual development:

- Backend modules were built and tested one at a time in dependency order — Authentication first (since every other module requires it), followed by Project, Task, Comment, Notification, and Dashboard — with each module verified through Postman before the next began.
- The frontend followed the same incremental pattern: Auth and Dashboard were built and confirmed working end-to-end against the live API before Projects, then Tasks, then Comments and Notifications were added on top.
- This structure meant that even mid-way through development, a fully working (if smaller) system existed at every stage — useful given the fixed academic submission deadlines for Synopsis, Chapters 1–2, Chapters 3–4, and the Final Report.

- Defects were caught early and locally: for example, an AutoMapper registration error and a task-comment-count bug were each isolated to a single increment rather than requiring a search across the whole codebase, because only one module was under active change at a time.
- The Task Workflow Engine itself (Phase 6) was added as a further increment on top of the already-working CRUD system, without modifying any existing table or breaking any working endpoint: three additive database columns (AssignmentNote, a discriminated Type column on Comments, and a nullable RelatedTaskId on Notifications) and a set of new, narrowly-scoped endpoints were layered on top of the existing schema and services, reusing an audit-log table that had existed since the first migration but was previously unused. This is the incremental model in its most direct sense — extending, not rewriting.

Chapter 2: Software Analysis & Software Requirement Specification

2.1 Feasibility Study

2.1.1 Technical Feasibility

NexusFlow uses a mature, well-documented technology stack (ASP.NET Core, Angular, SQL Server) with extensive official documentation and community support. All required tools — Visual Studio 2022 Community, .NET 10 SDK, Node.js, Angular CLI, SQL Server Developer Edition, and SSMS — are freely available for student use. The technology stack was learned progressively through a structured five-week training curriculum prior to development, and the incremental development approach (Section 1.5.2) allowed technical risk to be addressed one module at a time rather than all at once. This is no longer a purely theoretical assessment: the Review Build (Release Candidate v1.0), including the full task workflow, was built, run locally, and demonstrated to the Project Guide, confirming the stack's feasibility in practice rather than on paper. The project is therefore technically feasible within the skill level achievable during the training period.

2.1.2 Economical Feasibility

All software used in development is free for student/educational use: Visual Studio 2022 Community, SQL Server Developer Edition, Angular and its open-source ecosystem, and Postman's free tier. No paid cloud hosting, third-party APIs, or commercial licenses are required to build, run, or demonstrate the system locally. The project therefore involves no direct monetary cost.

2.1.3 Operational Feasibility

The system is accessed through a standard web browser, requiring no specialized end-user training beyond familiarity with common web application patterns (forms, dashboards, navigation menus). Role-based access control means each user type sees only the actions relevant to their role, simplifying day-to-day operation. Since the target users (small organizations, startups, and student teams) already use browser-based tools for similar purposes, adoption does not require a significant behavioural change.

2.1.4 Other Feasibility Dimensions

- **Schedule Feasibility:** The functional modules were scoped to fit the six-week training window (with an additional documentation period), and the actual development — including the added Task Workflow Engine phase — followed the incremental schedule in Section 1.4.1 without major slippage.
- **Legal Feasibility:** All frameworks, libraries, and tools used (ASP.NET Core, Angular, Entity Framework Core, AutoMapper, FluentValidation, SQL Server Developer Edition) are used under their respective free/open-source or free-for-development licenses, with no legal or licensing obstacles to development or academic submission.

2.2 Description of Information System (Block Diagram)

The diagram below illustrates the end-to-end flow of a typical request through NexusFlow — from the user's action in the Angular UI, through JWT authentication and FluentValidation checks, into the Application service layer, through Entity Framework Core to SQL Server, and back.

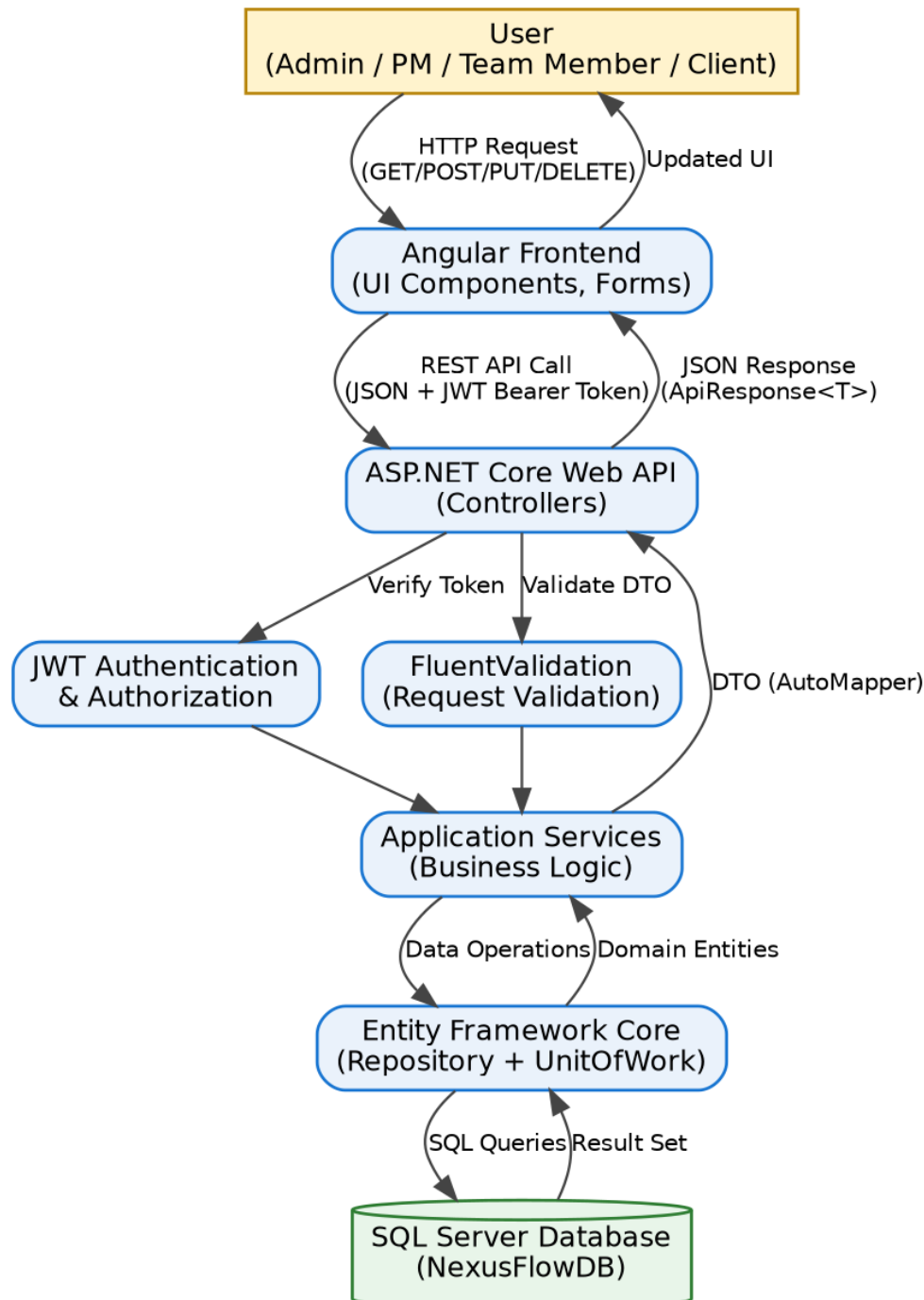


Figure 2.1: Block Diagram for NexusFlow Request Flow

2.2.1 Product Features

The following distinct processes (modules) were identified for NexusFlow:

1. Authentication & Authorization — registration, login, JWT issuance/refresh, role-based access.
2. User Management — listing and administering user accounts.
3. Organization Management — grouping projects and users under an organization entity.
4. Project Management — create, read, update, delete (CRUD) operations on projects, including status lifecycle and automatically-calculated completion progress.
5. Task Management — creation, priority levels, due dates, multi-assignee support, and an Assignment Note captured at assignment time.
6. Task Workflow Engine — status-transition actions — Accept Task and Update Status (optionally logging a Work Log note) — gated by whether the caller is the task's assignee, its creator, or a Project Manager/Admin.
7. Clarification Exchange — an assignee may Request Clarification on a task; only its creator or a Project Manager/Admin may Respond — both stored as typed entries in the same Comment table.
8. Activity Timeline — a chronological, automatically-written audit trail (created, assigned, accepted, clarification requested/added, status changed) surfaced per task.
9. Notification — automatic, clickable alerts on events such as task assignment, carrying a reference back to the related task, with read/unread tracking.
10. Dashboard & Reporting (mandatory Report Generation module) — aggregated statistics (total/active projects, task counts by status, unread notifications), a personal “My Tasks” view, project completion progress, and a recent activity feed.

2.2.2 Input / Output Data Elements

Input / Output	Data Elements
Register Request (Input)	FullName, Email, Password, ConfirmPassword, PhoneNumber
Login Request (Input)	Email, Password
Auth Response (Output)	UserId, FullName, Email, Role, AccessToken, RefreshToken, AccessTokenExpiry
Create Project (Input)	Name, Description, StartDate, EndDate, OrganizationId
Project Details (Output)	Id, Name, Status, StartDate, EndDate, MemberCount, TaskCount, CompletedTaskCount, CreatedAt
Create Task (Input)	Title, Description, Priority, DueDate, ProjectId, AssigneeIds, AssignmentNote
Task Details (Output)	Id, Title, Status, Priority, ProjectName, AssigneeNames, CommentCount, SubTaskCount, AssignmentNote
Accept Task (Input)	TaskId (route parameter only — no request body)
Request Clarification (Input)	Message
Clarification Response (Input)	Message
Update Task Status (Input)	Status, Note (optional work-log text)
Create Comment (Input)	Content, TaskId
Comment Details (Output)	Id, Content, UserName, CreatedAt, Type (Comment / Clarification / ClarificationResponse / WorkLog)
Task Activity (Output)	Action, OldValue, NewValue, UserId, UserName, CreatedAt
Notification (Output)	Id, Title, Message, Type, IsRead, CreatedAt, RelatedTaskId
Dashboard Stats (Output)	TotalProjects, ActiveProjects, TotalTasks, CompletedTasks, PendingTasks, InProgressTasks, UnreadNotifications, RecentActivities

Table 2.1: Input/Output Data Elements

2.2.3 Procedures / Rules / Mathematical Relationships

- The following business rules govern how NexusFlow converts input into output:
- Password Security Rule: Passwords are never stored in plain text. On registration, Password is transformed via `BCrypt.HashPassword()` into `PasswordHash` before persistence; on login, the supplied password is checked with `BCrypt.Verify()` against the stored hash.
- Token Expiry Rule: `AccessTokenExpiry = CreatedAt + 60 minutes`; `RefreshToken.ExpiresAt = CreatedAt + 7 days`. A refresh request issued after the refresh token's expiry is rejected with HTTP 401.
- Task Status Transition Rule: A newly created task always starts at `Status = Todo`, regardless of what the client requests. From there, status changes only through two authorized paths: Accept Task (`Todo → InProgress`, assignee only) and Update Task Status (caller-supplied status, restricted to the task's creator, its assignees, or a Project Manager/Admin). The current build does not yet enforce which specific transitions are legal beyond this permission check — this is a documented limitation (Chapter 4).
- Clarification Rule: Request Clarification is permitted only for a task's assignee; Respond to Clarification is permitted only for the task's creator or a user whose global Role is Admin or ProjectManager. Both are persisted as Comment rows distinguished by Type (Clarification / ClarificationResponse) rather than in a separate table.
- Work Log Rule: If a Note is supplied with an Update Task Status request, it is persisted as a Comment row with `Type = WorkLog`, preserving a running history of progress notes rather than overwriting the task's Description.
- Activity Logging Rule: Every workflow action (task creation, assignment, acceptance, clarification requested/added, status change) writes one row to the AuditLog table — `EntityName`, `EntityId`, `Action`, `OldValue`, `NewValue`, `UserId`, `CreatedAt` — read back by the Task Activity endpoint in chronological order to render the Activity Timeline. No workflow event is ever edited in place.
- Notification Trigger Rule: A Notification row (`Type = TaskAssigned`) is automatically created for every user listed in `AssigneeIds` at the moment a task is created, and carries a `RelatedTaskId` so the frontend can navigate directly to that task when the notification is clicked.
- Dashboard Aggregation Formulas: `ActiveProjects = COUNT(Projects WHERE Status = Active)`; `CompletedTasks = COUNT(Tasks WHERE Status = Done)`; `InProgressTasks = COUNT(Tasks WHERE Status = InProgress)`; `PendingTasks = COUNT(Tasks WHERE Status = Todo)`; `UnreadNotifications = COUNT(Notifications WHERE UserId = current user AND IsRead = false)`.
- Project Progress Rule: `CompletedTaskCount` and `TaskCount` are computed per project (`CompletedTaskCount = COUNT(Tasks WHERE ProjectId = p.Id AND Status = Done)`) and returned on

both the project list and single-project endpoints, driving the progress bar shown on the Dashboard, Project Cards, and Project Details screen.

- **Soft Delete Rule:** Delete operations never remove a database row; they set `IsDeleted = true`. All read queries implicitly filter `WHERE IsDeleted = false`, preserving an audit trail.

2.2.4 Decision Table

The table below summarizes which of the four user roles may perform each key action (Y = allowed, N = not allowed):

Action	Admin	Project Manager	Team Member	Client
Create / Edit / Delete Project	Y	Y	N	N
Assign Task to a User	Y	Y	N	N
Accept Task ¹	N	N	Y	N
Request Clarification ¹	N	N	Y	N
Respond to Clarification ²	Y	Y	N	N
Update Task Status ²	Y	Y	Y	N
Add Task Comment	Y	Y	Y	N
View Activity Timeline	Y	Y	Y	N
View Dashboard / Reports	Y	Y	Y	Y
Manage Users	Y	N	N	N

Table 2.2: Role-Based Decision Table

¹ Also requires the acting user to be an assignee of that specific task, regardless of role. ² Also succeeds for the task's creator even when their role is Team Member.

2.3 Use Case Diagram

The diagram below shows the relationship between NexusFlow's four actors and the system's principal use cases, including the task-workflow actions introduced in the Review Build.

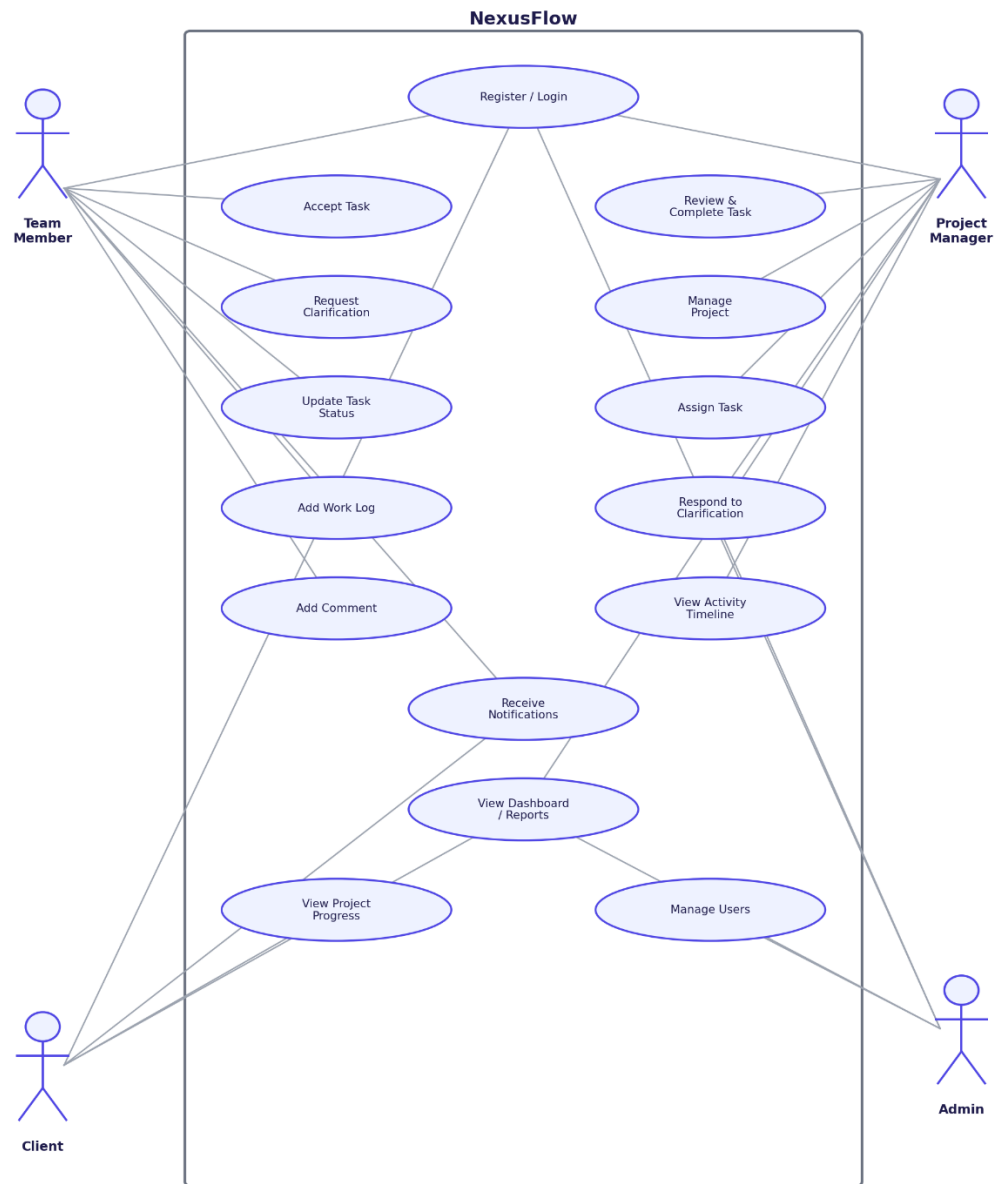


Figure 2.2: Use Case Diagram

Actor Descriptions

- Admin: Manages user accounts and organization-level settings; may also respond to clarification requests, and has full visibility into system-wide reports.
- Project Manager: Creates and manages projects, assigns tasks, responds to clarification requests, reviews and completes tasks, and monitors the Activity Timeline.
- Team Member: Registers/logs in, accepts assigned tasks, requests clarification when needed, updates task status with optional work-log notes, comments, and receives clickable notifications.
- Client: Views project progress and reports and receives status notifications, without edit access to projects or tasks.

2.4 Software Product Constraints

- The application requires network connectivity between the Angular client and the ASP.NET Core API (both running on localhost during development); it is not designed for offline use.
- An expired JWT access token requires either a silent refresh (via the stored refresh token) or a fresh login — there is no session that persists indefinitely without one of these.
- SQL Server must be running and reachable via the connection string in appsettings.json before the API will start correctly.
- The current build assumes a single Organization context; multi-tenant organization switching is not implemented (see Future Scope).
- Notifications are delivered via on-demand polling when the notification panel is opened, not push-based real-time delivery.
- The Update Task Status action does not currently validate that a requested transition is a legal next step in the workflow (for example, Todo directly to Done) — any status value supplied by an authorized caller is accepted. This is scoped as a near-term refinement (Chapter 4).
- Read access to an individual task (GetById), and the original Update/Delete task endpoints, do not yet verify that the requesting user is a member of that task's project — only the newer workflow actions (Accept, Update Status, Clarification) enforce this. Closing this gap is prioritized ahead of any new feature work.
- The Angular 20 frontend requires a modern evergreen browser (Chrome, Edge, Firefox) with JavaScript enabled; it is not designed to support legacy browsers.

Chapter 3: Software Project Analysis

3.1 Data Flow Diagram (DFD)

A Data Flow Diagram models how data moves through NexusFlow — between the user, the system's internal processes, and its data stores — without describing the internal implementation of each process. Three levels of increasing detail are presented below: a Context (Level 0) diagram showing NexusFlow as a single process, a Level 1 diagram decomposing it into its six major processes, and a Level 2 diagram further decomposing the Task Management process, since task handling — now including the full accept / clarify / status-transition workflow — is the most operationally complex module in the system.

3.1.1 Level 0 — Context Diagram

The context diagram treats NexusFlow as a single process and shows only the data exchanged with its external entity, the User (representing any of the four roles: Admin, Project Manager, Team Member, or Client). This top-level view is unchanged by the Task Workflow Engine, since the engine only adds detail inside the system, not new external data flows.

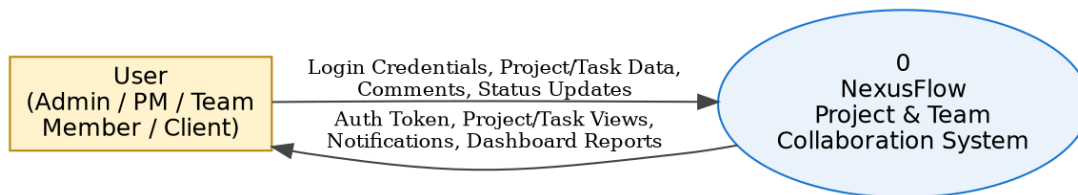


Figure 3.1: DFD Level 0 (Context Diagram)

3.1.2 Level 1 — Top-Level Decomposition

Level 1 decomposes the system into its six principal processes, each reading from and writing to its own data store. A seventh data store, D7 (AuditLogs), is introduced here — the table existed in the schema from the first migration but was inactive until the Task Workflow Engine began writing to it:

1.0 Authenticate User — validates credentials against D1 (Users) and issues a JWT.

2.0 Manage Projects — performs CRUD operations against D2 (Projects), including computing each project's completed/total task counts from D3 for the progress bar.

3.0 Manage Tasks — performs CRUD, assignment, and the full workflow (accept, clarify, status transitions) against D3 (Tasks), D6 (TaskAssignees), and D4 (Comments), writes every workflow event to D7 (AuditLogs), and triggers Process 5.0 on assignment, clarification, and status changes.

4.0 Manage Comments — reads/writes D4 (Comments, including clarification and work-log entries distinguished by Type) and looks up commenter names from D1 (Users).

5.0 Manage Notifications — writes to D5 (Notifications), tagging each with a RelatedTaskId where applicable, and delivers alerts back to the User.

6.0 Generate Dashboard — reads aggregated data from D2, D3, and D5 to compute the statistics and “My Tasks” view shown on the Dashboard.

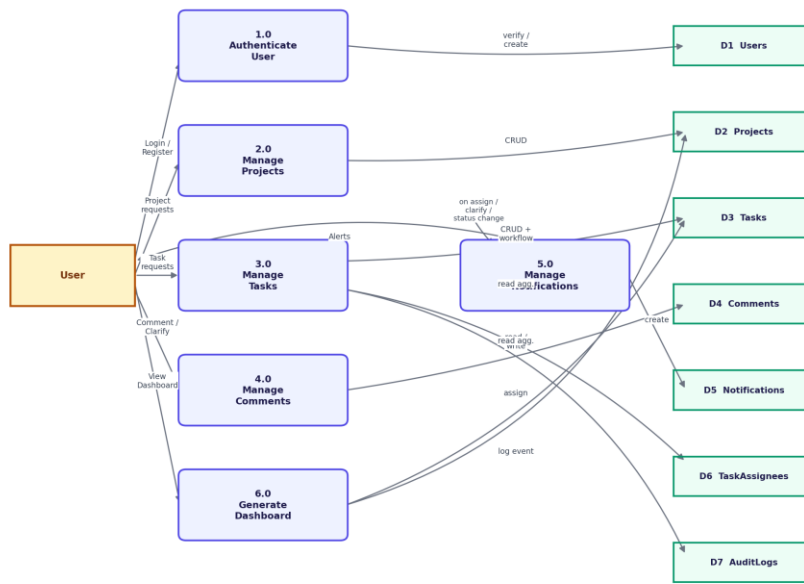


Figure 3.2: DFD Level 1

3.1.3 Level 2 — Decomposition of Process 3.0 (Manage Tasks)

Process 3.0 is further decomposed into seven sub-processes, reflecting the actual task lifecycle implemented by the Task Workflow Engine — an expansion from the four sub-processes of the original CRUD-only build:

3.1 Create Task — inserts a new row into D3 (Tasks) with Status fixed to Todo regardless of client input, then passes the new TaskId to 3.2.

3.2 Assign Task to Users — inserts rows into D6 (TaskAssignees) for each selected user, creates a corresponding alert in D5 (Notifications), and logs an “Assigned” event to D7.

3.3 Accept Task — verifies the caller is a D6 assignee and the task is still Todo, then updates D3.Status to InProgress and logs an “Accepted” event to D7. Restricted to the assignee only.

3.4 Request / Respond to Clarification — inserts a typed row into D4 (Type = Clarification or ClarificationResponse), notifies the other party via Process 5.0, and logs the event to D7. Request is restricted to an assignee; Respond is restricted to the task's creator or a Project Manager/Admin.

3.5 Update Task Status + Work Log — updates D3.Status to the caller-supplied value; if a work-log note is supplied, inserts it into D4 as Type = WorkLog; logs a “StatusChanged” event to D7. Restricted to the task's creator, its assignees, or a Project Manager/Admin.

3.6 View Activity Timeline — a read-only sub-process that queries D7 for all rows matching the given TaskId and returns them in chronological order.

3.7 Delete Task (Soft Delete) — sets IsDeleted = true on the D3 row rather than physically removing it, preserving the audit trail.

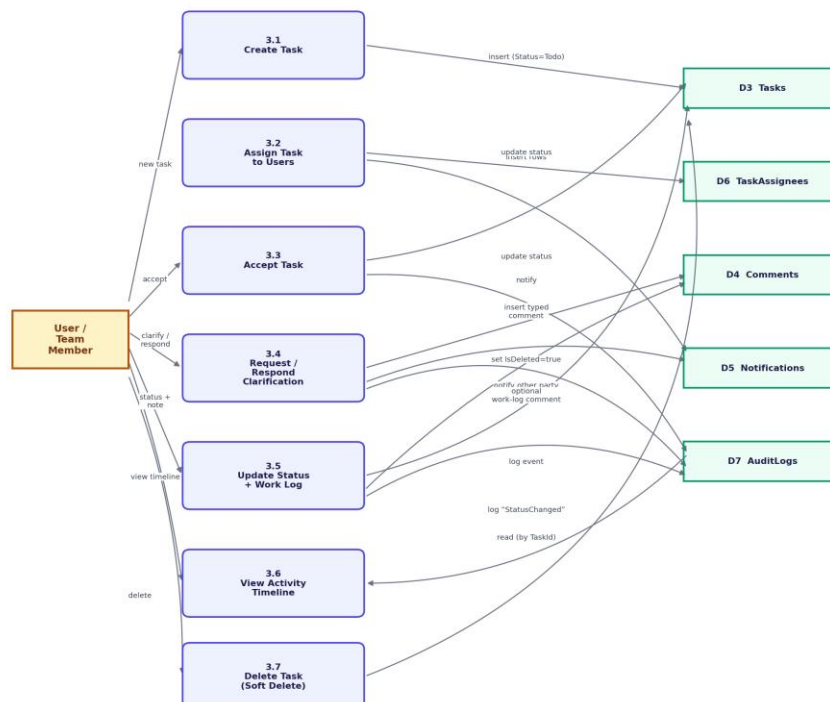
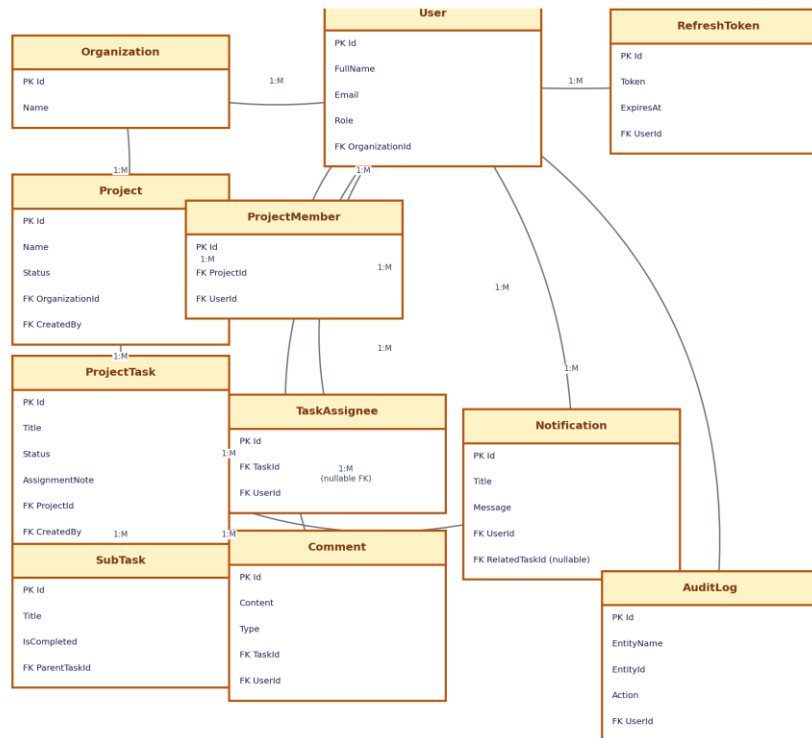


Figure 3.3: DFD Level 2 (Expansion of Process 3.0 — Manage Tasks)

3.2 Entity-Relationship Diagram

The Entity-Relationship diagram below models the ten most structurally significant entities of NexusFlow's fourteen-table schema (Organization, User, Project, ProjectMember, ProjectTask, TaskAssignee, SubTask, Comment, Notification, and RefreshToken) and the relationships between them, plus AuditLog — now shown explicitly since the Task Workflow Engine activated it. Department, Team, and TeamMember exist in the schema

to support future organizational hierarchy features but are omitted here for clarity, as they do not yet participate in the core project/task workflow.



Department, Team, and TeamMember exist in the schema for future organizational hierarchy features and are omitted above for clarity.

Figure 3.4: Entity-Relationship Diagram

Key Relationships

- An Organization has many Users and many Projects (1:M) — every user and project belongs to exactly one organization.
- A Project has many ProjectMembers and many ProjectTasks (1:M); a User can, through ProjectMember, belong to many projects — a many-to-many relationship between User and Project realized via the ProjectMember junction table.
- A ProjectTask has many TaskAssignees (1:M), which similarly realizes a many-to-many relationship between ProjectTask and User — a task can have multiple assignees, and a user can be assigned to multiple tasks.

- A ProjectTask has many SubTasks and many Comments (1:M); each Comment also belongs to exactly one User (the commenter) and carries a Type distinguishing a plain comment from a clarification request, a clarification response, or a work-log entry.
- A User has many Notifications and many RefreshTokens (1:M) — every login issues a new refresh token, and old tokens are revoked (not deleted) on re-login, preserving history.
- A Notification optionally references a ProjectTask via a nullable RelatedTaskId (M:1, new in this build) — this is what lets the frontend navigate directly to the task a notification is about when it is clicked.
- A User has many AuditLog entries (1:M, newly activated) — every workflow action performed by a user writes one row here, forming the Activity Timeline read back by Process 3.6.

3.3 Database Specification

The six tables below are the core operational tables of NexusFlow's fourteen-table schema. Field sizes shown are the actual constraints configured in NexusFlowDbContext (via Entity Framework Core's Fluent API); fields without a stated size are stored as nvarchar(max) or their natural SQL type

Table: Users

Field Name	Type	Size	Description
Id	int	—	Primary key, identity
FullName	nvarchar	100	User's display name
Email	nvarchar	150	Login identifier, unique per user
PasswordHash	nvarchar	max	BCrypt hash of the password (never plain text)
Role	int	—	Enum: Admin=1, ProjectManager=2, TeamMember=3, Client=4
PhoneNumber	nvarchar	max, nullable	Optional contact number
OrganizationId	int	nullable	Foreign key to Organizations
CreatedAt / UpdatedAt	datetime2	—	Audit timestamps (inherited from BaseEntity)
IsDeleted / IsActive	bit	—	Soft-delete and active-status flags

Table 3.1: Field Specification — Users

Table: Projects

Field Name	Type	Size	Description
Id	int	—	Primary key, identity
Name	nvarchar	150	Project name
Description	nvarchar	max, nullable	Free-text project description
Status	int	—	Enum: Planning=1, Active=2, OnHold=3, Completed=4, Cancelled=5
StartDate / EndDate	datetime2	nullable	Planned project timeline
OrganizationId	int	—	Foreign key to Organizations
CreatedBy	int	—	Foreign key to Users (creator)

*Table 3.2: Field Specification — Projects***Table: ProjectTasks**

Field Name	Type	Size	Description
Id	int	—	Primary key, identity
Title	nvarchar	200	Task title
Description	nvarchar	max, nullable	Free-text task description
Status	int	—	Enum: Todo=1, InProgress=2, InReview=3, Done=4, Cancelled=5
Priority	int	—	Enum: Low=1, Medium=2, High=3, Critical=4
DueDate	datetime2	nullable	Task deadline
AssignmentNote	nvarchar	2000, nullable	New in this build — manager's note captured at assignment time, shown on the Task Details page
ProjectId	int	—	Foreign key to Projects
CreatedBy	int	—	Foreign key to Users (creator)

Table 3.3: Field Specification — ProjectTasks

Table: Comments

Field Name	Type	Size	Description
Id	int	—	Primary key, identity
Content	nvarchar	1000	Comment / clarification / work-log text
Type	int	—, default 1	New in this build — Enum: Comment=1, Clarification=2, ClarificationResponse=3, WorkLog=4. Discriminates the four kinds of entry stored in this one table.
TaskId	int	—	Foreign key to ProjectTasks
UserId	int	—	Foreign key to Users (author)

*Table 3.4: Field Specification — Comments***Table: Notifications**

Field Name	Type	Size	Description
Id	int	—	Primary key, identity
Title	nvarchar	200	Short notification heading
Message	nvarchar	500	Notification body text
IsRead	bit	—	Read/unread flag, defaults to false
Type	int	—	Enum: TaskAssigned=1, TaskUpdated=2, ProjectCreated=3, CommentAdded=4, DeadlineApproaching=5, MentionedInComment=6, ClarificationRequested=7, ClarificationResponded=8
RelatedTaskId	int	nullable	New in this build — foreign key to ProjectTasks; enables click-through navigation from a notification straight to its task
UserId	int	—	Foreign key to Users (recipient)

Table 3.5: Field Specification — Notification

Table: AuditLogs

Newly activated in this build — the table existed in the schema since the first migration but was not written to until the Task Workflow Engine began logging events. It backs the Activity Timeline shown on the Task Details page.

Field Name	Type	Size	Description
Id	int	—	Primary key, identity
EntityName	nvarchar	100	Name of the entity type the event relates to (e.g. “ProjectTask”)
EntityId	int	—	Id of the specific row the event relates to
Action	nvarchar	50	Event name, e.g. “Created”, “Accepted”, “StatusChanged”
OldValue / NewValue	nvarchar	max, nullable	Before/after values, where applicable (e.g. previous and new status)
UserId	int	—	Foreign key to Users (who performed the action)

Table 3.6: Field Specification — AuditLogs

The remaining eight tables (Organizations, Departments, Teams, TeamMembers, ProjectMembers, SubTasks, TaskAssignees, RefreshTokens) follow the same pattern — an int identity primary key, foreign keys to their parent entities, and the CreatedAt/UpdatedAt/CreatedBy/UpdatedBy/IsDeleted/IsActive audit columns inherited from the common BaseEntity base class.

3.4 Validation Specification

Input validation is enforced server-side using FluentValidation, wired through a global ValidationFilter that runs automatically before every controller action — no controller or service method needs to call a validator manually. If validation fails, the API returns HTTP 400 with a list of human-readable error messages before any business logic executes.

Input Screen / Action (DTO)	Field	Validation Rule
Register	FullName	Required, max 150 characters
Register	Email	Required, valid email format, max 256 characters
Register	Password	Required, min 8 characters, at least one uppercase, one lowercase, one digit
Register	ConfirmPassword	Required, must equal Password
Login	Email / Password	Both required
Create Project	Name	Required, max 200 characters
Create Project	EndDate	Must be on or after StartDate, when both are supplied
Update Project	Status	Must be a valid ProjectStatus enum value
Create Task	Title	Required, max 200 characters
Create Task	Priority	Must be a valid TaskPriority enum value
Create Task	AssigneeIds	Each Id, if present, must be greater than 0
Create Comment	Content	Required, max 1000 characters
Request Clarification	Message	Required (“Please describe what needs clarification.”), max 1000 characters
Respond to Clarification	Message	Required (“Response cannot be empty.”), max 1000 characters
Update Task Status	Status	Must be a valid TaskStatus enum value
Update Task Status	Note	Optional; when supplied, max 2000 characters

Table 3.7: Validation Specification

3.5 Input Design

Input screens use Angular Reactive Forms with Angular Material components, giving inline, field-level error messages the moment a field is touched and left invalid, rather than a single generic error after submission.

Screen	Input Controls Used
Register / Login	Text input, password input, inline validation messages
Create / Edit Project	Text input, multi-line textarea, status dropdown (edit only), date pickers
Create / Edit Task	Text input, textarea, priority dropdown, status dropdown (edit only), multi-select dropdown (assignees), date picker, assignment-note textarea
Task Details — Workflow Actions	Accept Task button (assignee only), Request/Respond Clarification text areas, status action buttons with an optional work-log note field
Add Comment	Single-line text input with Enter-to-submit

Table 3.8: Input Design Summary

3.6 Output Design

Output is presented as card-based layouts throughout, consistent with the visual language of the reference products studied (Jira, Trello, Asana) rather than dense tabular grids, which better suits the four distinct user roles browsing on both desktop and mobile widths.

Screen	Output Elements
Dashboard	Six stat cards (Total/Active Projects, Total/Completed/In-Progress Tasks, Unread Notifications) + a personal “My Tasks” list + Recent Activity feed
Projects List	One card per project: name, status chip, description, member/task counts, a completion progress bar, Edit/Delete/View Tasks actions
Tasks List	One card per task: title, priority chip, description, due date, status, comment thread, Edit/Delete actions
Task Details	Description, Assignment Note, status, priority, due date, assignees, comment thread, work-log history, and a chronological Activity Timeline, plus role-aware action buttons (Accept, Request/Respond Clarification, Update Status)
Notifications	Dropdown panel from the bell icon: title, message, timestamp, unread highlight, mark-as-read and navigate-to-task on click

Table 3.9: Output Design Summary

3.7 Code Design and Development

The backend follows Clean Architecture, split into four projects with a strict dependency direction — API depends on Application, Application depends on Domain, and Infrastructure implements interfaces defined in Domain, so business logic never depends on EF Core or ASP.NET Core directly:

- NexusFlow.Domain — entities, enums, and repository interfaces only. No dependency on any other project.
- NexusFlow.Application — DTOs, service interfaces and implementations (business logic), AutoMapper profiles, FluentValidation validators. Depends only on Domain.
- NexusFlow.Infrastructure — EF Core DbContext, the generic Repository and UnitOfWork implementations. Implements Domain's interfaces.
- NexusFlow.API — controllers, middleware, and DI wiring in Program.cs. The only layer aware of HTTP concerns.

Every service method follows the same pattern — fetch via the generic repository, apply business rules, map to DTO via AutoMapper, and return a uniform ApiResponse<T> wrapper. The snippet below (from TaskService) illustrates task creation as actually implemented:

```
public async Task<ApiResponse<TaskDto>> CreateAsync(CreateTaskDto dto, int userId)
{
    var task = _mapper.Map<ProjectTask>(dto);
    task.Status = TaskStatus.TODO; // business rule: always starts at TODO
    task.CreatedBy = userId;

    await _unitOfWork.Repository<ProjectTask>().AddAsync(task);
    await _unitOfWork.SaveChangesAsync();

    foreach (var assigneeId in dto.AssigneeIds)
    {
        await _unitOfWork.Repository<TaskAssignee>().AddAsync(
            new TaskAssignee { TaskId = task.Id, UserId = assigneeId, CreatedBy = userId });
        await _notificationService.CreateNotificationAsync(
            assigneeId, "New Task Assigned", $"...{task.Title}",
            NotificationType.TaskAssigned, task.Id);
    }
    return ApiResponse<TaskDto>.Created(await
        EnrichAsync(_mapper.Map<TaskDto>(task), task));
}
```

The Task Workflow Engine's Accept Task action follows the same repository/business-rule/response pattern, but illustrates how each workflow action independently re-checks both the caller's relationship to the task and the task's current state before making any change — this is the mechanism behind the role-based rules summarized in Table 2.2:

```
public async Task<ApiResponse<TaskDto>> AcceptTaskAsync(int id, int userId)
{
    var task = (await _unitOfWork.Repository<ProjectTask>()
        .FindAsync(t => t.Id == id && !t.IsDeleted)).FirstOrDefault();
    if (task == null)
        return ApiResponse<TaskDto>.Fail("Task not found.", 404);

    var assignees = await _unitOfWork.Repository<TaskAssignee>()
        .FindAsync(a => a.TaskId == id && !a.IsDeleted);
    if (!assignees.Any(a => a.UserId == userId))
        return ApiResponse<TaskDto>.Fail(
            "Only an assignee can accept this task.", 403);

    if (task.Status != TaskStatus.TODO)
        return ApiResponse<TaskDto>.Fail(
            "This task has already been accepted or moved past TODO.", 400);

    task.Status = TaskStatus.InProgress;
    task.UpdatedBy = userId;
    _unitOfWork.Repository<ProjectTask>().Update(task);
    await _unitOfWork.SaveChangesAsync();
    // ... AuditLog entry written here, see Section 2.2.3
}
```

On the frontend, Angular 20 standalone components (no NgModules) are organized by feature (features/auth, features/projects, features/tasks, including the new features/tasks/task-details and features/tasks/my-tasks) with a core/ folder for cross-cutting services, models, guards, and an HTTP interceptor that attaches the JWT to every outgoing request automatically. Component state uses Angular's signal() API rather than manual change detection, and all authenticated routes are lazy-loaded (loadComponent) and nested under a single Shell layout component so navigation and notifications are consistent across every page.

Chapter 4: Conclusion and Future Scope

NexusFlow was built to address a genuine gap in how small organizations and student teams manage projects — the fragmentation of task tracking, communication, and reporting across disconnected tools such as spreadsheets, chat applications, and email. Across the training period, the system was developed incrementally: a Clean Architecture ASP.NET Core (.NET 10) backend with eight functional modules, secured by JWT authentication and role-based authorization, paired with an Angular 20 frontend built on standalone components and signal-based state. Beyond basic CRUD, the system now implements a genuine task workflow — assignment with a note, acceptance, clarification request/response, status transitions with optional work-log entries, and a chronological Activity Timeline — verified end-to-end against a live SQL Server database as a Review Build (Release Candidate v1.0) demonstrated to the Project Guide, rather than left as an untested design on paper.

4.1 Limitations of the System

- The system currently supports a single Organization context; there is no UI for creating or switching between multiple organizations, so all projects belong to one demo organization.
- Notifications are delivered via on-demand polling (fetched when the bell icon is opened), not push-based real-time delivery — a genuinely new notification is not shown until the panel is reopened.
- No automated unit or integration test suite exists; verification was performed manually through Postman (backend) and the browser (frontend) at each development increment, including the Review Build.
- File and document attachment (for projects or tasks) is not implemented in the current build.
- The Calendar & Meetings and Admin Panel modules described in the original project scope were de-prioritized in favor of completing the core Project/Task/Workflow/Comment/Notification loop within the training timeline.
- Client-role users have read-only dashboard/report access; a dedicated milestone-approval workflow for Clients is designed into the role matrix (Chapter 2, Table 2.2) but not yet built.
- The Update Task Status action does not yet validate that a requested transition is a legal next step in the workflow (for example, Todo directly to Done) — any status value supplied by an authorized caller is currently accepted.
- Read access to an individual task (GetById), and the original Update/Delete task endpoints, do not yet verify that the requesting user is a member of that task's project — only the newer workflow actions (Accept, Update Status, Clarification) enforce this membership check.

4.2 Future Scope for Modification

1. Enforce a formal status-transition state machine on Update Task Status, and extend the project-membership authorization check already used by the workflow actions to the original GetById/Update/Delete task endpoints.
2. Real-time notifications and live updates using SignalR, replacing the current polling mechanism with instant push delivery.
3. File attachment support for tasks and projects via cloud storage integration (e.g., Azure Blob Storage).
4. Organization Management UI, allowing an Admin to create and manage multiple organizations rather than relying on a single seeded row.
5. Calendar and meeting scheduling module for coordinating team availability and deadlines.
6. Advanced analytics with visual charts (Chart.js) — burn-down charts, team workload distribution, and historical velocity, including a chart view of Activity Timeline data already being collected.
7. Automated testing (xUnit for the backend, Jasmine/Karma for the frontend) to replace the current manual verification workflow.
8. Mobile support via a Progressive Web App (PWA) build of the existing Angular frontend.
9. Integration with third-party developer tools such as GitHub, linking code commits directly to tasks.